

Experiment No: 10

Title

Write a Python Program to Implement A* Algorithm

Aim

To develop a Python program to implement the A* search algorithm for finding the shortest path between a start node and a goal node.

Objective

To implement the A* search algorithm using a priority queue and heuristic function to determine the optimal path from the source node to the destination node.

Algorithm

Step 1: Start the program.

Step 2: Define the graph and heuristic values.

Step 3: Initialize the open list with the start node.

Step 4: Select the node with the lowest $f(n) = g(n) + h(n)$.

Step 5: Expand the current node and calculate the cost of its neighbours.

Step 6: Update the cost and parent of each neighbour if a shorter path is found.

Step 7: Repeat until the goal node is reached or the open list becomes empty.

Step 8: Display the shortest path and total cost.

Step 9: Stop the program.

Program

```
import heapq
```

```
graph = {  
    'A': [('B', 2), ('C', 4)],  
    'B': [('D', 3), ('E', 5)],  
    'C': [('F', 2)],
```

```
'D': [('G', 4)],  
'E': [('G', 2)],  
'F': [('G', 3)],  
'G': []  
}
```

```
heuristic = {  
    'A': 8,  
    'B': 6,  
    'C': 5,  
    'D': 3,  
    'E': 2,  
    'F': 2,  
    'G': 0  
}
```

```
def astar(start, goal):
```

```
    priority_queue = []  
    heapq.heappush(priority_queue, (0, start))
```

```
    parent = {}  
    cost = {}
```

```
    parent[start] = None  
    cost[start] = 0
```

```
    while priority_queue:
```

```
current = heapq.heappop(priority_queue)[1]
```

```
if current == goal:
```

```
    break
```

```
for neighbour, weight in graph[current]:
```

```
    new_cost = cost[current] + weight
```

```
    if neighbour not in cost or new_cost < cost[neighbour]:
```

```
        cost[neighbour] = new_cost
```

```
        priority = new_cost + heuristic[neighbour]
```

```
        heapq.heappush(priority_queue, (priority, neighbour))
```

```
        parent[neighbour] = current
```

```
path = []
```

```
node = goal
```

```
while node is not None:
```

```
    path.append(node)
```

```
    node = parent[node]
```

```
path.reverse()
```

```
return path, cost[goal]
```

```
start = input("Enter Start Node : ").upper()
```

```
goal = input("Enter Goal Node : ").upper()
```

```
path, total_cost = astar(start, goal)
```

```
print("\nShortest Path :", " -> ".join(path))
```

```
print("Total Cost   :", total_cost)
```

Sample Input

Enter Start Node : A

Enter Goal Node : G

Sample Output

```
Enter Start Node : A
```

```
Enter Goal Node   : G
```

```
Shortest Path : A -> B -> D -> G
```

```
Total Cost      : 9
```

Result

Thus, the Python program to implement the A* algorithm was executed successfully, and the shortest path along with the minimum cost was obtained correctly.

